

Spotify Recommender System

This is a Jupyter notebook which features the EDA, preprocessing, model development, and evaluation & explainability of the Spotify tracks recommender which supplements the c.20-minute recorded presentation.

```
In [60]: # 1. IMPORT LIBRARIES
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import GroupKFold, train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PowerTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_squared_error, ndcg_score
import shap

import tensorflow as tf
from tensorflow.keras import layers, models, callbacks

import reportlab

# Set Seed for Later Models
SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

# Set Seaborn Theme
sns.set_theme(style = "whitegrid")

In [ ]: # 2. LOAD & INSPECT DATA
spotify = pd.read_csv("/Users/jaselliawilliams/Desktop/MSc Data Science/M5 - Machine Learning/Unit 11/Spotify Assign

print("Spotify Track Data:", spotify.shape) # Confirms 114,000 rows and 21 columns (features)
print(spotify.head(10))
print(spotify.info())
print(spotify.describe())
```

```
print(spotify.isnull().sum())

# Check For Any Duplicates
print(spotify.duplicated().sum()) # no need to drop any rows

# Data Cleaning

# Drop Rows with Missing Key Fields
spotify = spotify.dropna(subset = ["artists", "album_name", "track_name"])
print(spotify.shape) # 113,999 rows remaining

# Sanity Filters - Drop Entries with Unrealistic/Erroneous Metrics
spotify = spotify[spotify["duration_ms"] > 0]
spotify = spotify[(spotify["tempo"] > 0) & (spotify["popularity"] >= 0)]

spotify = spotify.reset_index(drop = True)
print(spotify.shape)
print(spotify.info()) # final sense check before EDA

audio_features = ["danceability", "energy", "loudness", "speechiness", "acousticness",
                  "instrumentalness", "liveness", "valence", "tempo"]

print(spotify[audio_features + ["popularity"]].describe().T)
```

Spotify Track Data: (114000, 21)

	Unnamed: 0	track_id	artists \
0	0	5Su0ikwiRyPMVoIQDJUgSV	Gen Hoshino
1	1	4qPNDBW1i3p13qLCt0Ki3A	Ben Woodward
2	2	1iJBSr7s7jYXzM8EGcbK5b	Ingrid Michaelson;ZAYN
3	3	6lfxq3CG4xtTiEg7opyCyx	Kina Grannis
4	4	5vjLSffimiIP26QG5WcN2K	Chord Overstreet
5	5	01MV0l9KtVTNfFiBU9I7dc	Tyrone Wells
6	6	6Vc5wAMmXdKIAM7WUoEb7N	A Great Big World;Christina Aguilera
7	7	1EzrE0XmMH3G43AXT1y7pA	Jason Mraz
8	8	0IktbUcnAGrvD03AWnz3Q8	Jason Mraz;Colbie Caillat
9	9	7k9GuJYLp2AzqokyEdwEw2	Ross Copperman

	album_name \
0	Comedy
1	Ghost (Acoustic)
2	To Begin Again
3	Crazy Rich Asians (Original Motion Picture Sou...
4	Hold On
5	Days I Will Remember
6	Is There Anybody Out There?
7	We Sing. We Dance. We Steal Things.
8	We Sing. We Dance. We Steal Things.
9	Hunger

	track_name	popularity	duration_ms	explicit \
0	Comedy	73	230666	False
1	Ghost - Acoustic	55	149610	False
2	To Begin Again	57	210826	False
3	Can't Help Falling In Love	71	201933	False
4	Hold On	82	198853	False
5	Days I Will Remember	58	214240	False
6	Say Something	74	229400	False
7	I'm Yours	80	242946	False
8	Lucky	74	189613	False
9	Hunger	56	205594	False

	danceability	energy	...	loudness	mode	speechiness	acousticness \
0	0.676	0.4610	...	-6.746	0	0.1430	0.0322
1	0.420	0.1660	...	-17.235	1	0.0763	0.9240
2	0.438	0.3590	...	-9.734	1	0.0557	0.2100

3	0.266	0.0596	...	-18.515	1	0.0363	0.9050
4	0.618	0.4430	...	-9.681	1	0.0526	0.4690
5	0.688	0.4810	...	-8.807	1	0.1050	0.2890
6	0.407	0.1470	...	-8.822	1	0.0355	0.8570
7	0.703	0.4440	...	-9.331	1	0.0417	0.5590
8	0.625	0.4140	...	-8.700	1	0.0369	0.2940
9	0.442	0.6320	...	-6.770	1	0.0295	0.4260

	instrumentalness	liveness	valence	tempo	time_signature	track_genre
0	0.000001	0.3580	0.7150	87.917	4	acoustic
1	0.000006	0.1010	0.2670	77.489	4	acoustic
2	0.000000	0.1170	0.1200	76.332	4	acoustic
3	0.000071	0.1320	0.1430	181.740	3	acoustic
4	0.000000	0.0829	0.1670	119.949	4	acoustic
5	0.000000	0.1890	0.6660	98.017	4	acoustic
6	0.000003	0.0913	0.0765	141.284	3	acoustic
7	0.000000	0.0973	0.7120	150.960	4	acoustic
8	0.000000	0.1510	0.6690	130.088	4	acoustic
9	0.004190	0.0735	0.1960	78.899	4	acoustic

[10 rows x 21 columns]

<class 'pandas.DataFrame'>

RangeIndex: 114000 entries, 0 to 113999

Data columns (total 21 columns):

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	114000 non-null	int64
1	track_id	114000 non-null	str
2	artists	113999 non-null	str
3	album_name	113999 non-null	str
4	track_name	113999 non-null	str
5	popularity	114000 non-null	int64
6	duration_ms	114000 non-null	int64
7	explicit	114000 non-null	bool
8	danceability	114000 non-null	float64
9	energy	114000 non-null	float64
10	key	114000 non-null	int64
11	loudness	114000 non-null	float64
12	mode	114000 non-null	int64
13	speechiness	114000 non-null	float64
14	acousticness	114000 non-null	float64

```

15  instrumentality  114000 non-null  float64
16  liveness         114000 non-null  float64
17  valence          114000 non-null  float64
18  tempo            114000 non-null  float64
19  time_signature    114000 non-null  int64
20  track_genre       114000 non-null  str

```

dtypes: bool(1), float64(9), int64(6), str(5)

memory usage: 17.5 MB

None

	Unnamed: 0	popularity	duration_ms	danceability \
count	114000.000000	114000.000000	1.140000e+05	114000.000000
mean	56999.500000	33.238535	2.280292e+05	0.566800
std	32909.109681	22.305078	1.072977e+05	0.173542
min	0.000000	0.000000	0.000000e+00	0.000000
25%	28499.750000	17.000000	1.740660e+05	0.456000
50%	56999.500000	35.000000	2.129060e+05	0.580000
75%	85499.250000	50.000000	2.615060e+05	0.695000
max	113999.000000	100.000000	5.237295e+06	0.985000

	energy	key	loudness	mode \
count	114000.000000	114000.000000	114000.000000	114000.000000
mean	0.641383	5.309140	-8.258960	0.637553
std	0.251529	3.559987	5.029337	0.480709
min	0.000000	0.000000	-49.531000	0.000000
25%	0.472000	2.000000	-10.013000	0.000000
50%	0.685000	5.000000	-7.004000	1.000000
75%	0.854000	8.000000	-5.003000	1.000000
max	1.000000	11.000000	4.532000	1.000000

	speechiness	acousticness	instrumentality	liveness \
count	114000.000000	114000.000000	114000.000000	114000.000000
mean	0.084652	0.314910	0.156050	0.213553
std	0.105732	0.332523	0.309555	0.190378
min	0.000000	0.000000	0.000000	0.000000
25%	0.035900	0.016900	0.000000	0.098000
50%	0.048900	0.169000	0.000042	0.132000
75%	0.084500	0.598000	0.049000	0.273000
max	0.965000	0.996000	1.000000	1.000000

	valence	tempo	time_signature
count	114000.000000	114000.000000	114000.000000

```

mean      0.474068    122.147837    3.904035
std       0.259261     29.978197    0.432621
min       0.000000     0.000000    0.000000
25%      0.260000     99.218750    4.000000
50%      0.464000    122.017000    4.000000
75%      0.683000    140.071000    4.000000
max       0.995000    243.372000    5.000000

```

```

Unnamed: 0      0
track_id        0
artists         1
album_name      1
track_name      1
popularity      0
duration_ms     0
explicit        0
danceability    0
energy          0
key             0
loudness        0
mode            0
speechiness     0
acousticness    0
instrumentalness 0
liveness       0
valence         0
tempo          0
time_signature  0
track_genre     0

```

```
dtype: int64
```

```
0
```

```
(113999, 21)
```

```
(113842, 21)
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 113842 entries, 0 to 113841
```

```
Data columns (total 21 columns):
```

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	113842 non-null	int64
1	track_id	113842 non-null	str
2	artists	113842 non-null	str
3	album_name	113842 non-null	str

```

4  track_name      113842 non-null str
5  popularity      113842 non-null int64
6  duration_ms     113842 non-null int64
7  explicit        113842 non-null bool
8  danceability    113842 non-null float64
9  energy          113842 non-null float64
10 key            113842 non-null int64
11 loudness        113842 non-null float64
12 mode           113842 non-null int64
13 speechiness     113842 non-null float64
14 acousticness    113842 non-null float64
15 instrumentalness 113842 non-null float64
16 liveness        113842 non-null float64
17 valence         113842 non-null float64
18 tempo          113842 non-null float64
19 time_signature  113842 non-null int64
20 track_genre     113842 non-null str

```

dtypes: bool(1), float64(9), int64(6), str(5)

memory usage: 17.5 MB

None

	count	mean	std	min	25%	\
danceability	113842.0	0.567582	0.172380	0.05130	0.4560	
energy	113842.0	0.642098	0.250794	0.00002	0.4730	
loudness	113842.0	-8.239872	4.993665	-46.59100	-10.0020	
speechiness	113842.0	0.084769	0.105759	0.02210	0.0359	
acousticness	113842.0	0.314656	0.332338	0.00000	0.0169	
instrumentalness	113842.0	0.155369	0.308893	0.00000	0.0000	
liveness	113842.0	0.213221	0.189929	0.00925	0.0980	
valence	113842.0	0.474720	0.258841	0.00000	0.2610	
tempo	113842.0	122.316149	29.653539	30.20000	99.4295	
popularity	113842.0	33.233367	22.316078	0.00000	17.0000	

	50%	75%	max
danceability	0.580000	0.69500	0.985
energy	0.685000	0.85400	1.000
loudness	-6.998000	-5.00000	4.532
speechiness	0.049000	0.08460	0.965
acousticness	0.168000	0.59700	0.996
instrumentalness	0.000041	0.04770	1.000
liveness	0.132000	0.27300	1.000
valence	0.464000	0.68300	0.995

tempo	122.023000	140.07775	243.372
popularity	34.000000	50.00000	100.000

```
In [57]: # 3a. UNIVARIATE ANALYSIS

# Distribution Plots - Numeric Variables
num_vars = spotify.select_dtypes(include = ['int', 'float'])

fig, axes = plt.subplots(5, 3, figsize = (18, 18))
fig.suptitle("Distribution of Numeric Variables", fontsize = 16, fontweight = 'bold')

for ax, col in zip(axes.flatten(), num_vars):
    ax.hist(spotify[col], bins = 30, color = "green", edgecolor = "black", alpha = 0.8)
    ax.set_title(col)
    ax.set_xlabel(col)
    ax.set_ylabel('Count')

plt.tight_layout()
# plt.savefig("Fig1_Numeric_Vars_Plots.png", dpi = 120)
plt.show()

# Distribution Plots - Audio Features
fig, axes = plt.subplots(3, 3, figsize = (15, 12))
fig.suptitle("Distribution of Audio Features", fontsize = 16, fontweight = 'bold')
for ax, col in zip(axes.flat, audio_features):
    sns.histplot(spotify[col], kde = True, ax = ax, color = "green")
    ax.set_title(col)
plt.tight_layout()
# plt.savefig("Fig2_Audio_Features_Plots.png", dpi = 120)
plt.show()

# Individual Plot for Popularity
plt.figure(figsize = (8, 4))
sns.histplot(spotify["popularity"], bins = 40, kde = True, color = "green")
plt.title("Distribution of Popularity", fontsize = 14, fontweight = 'bold')
# plt.savefig("Fig2.5__popularity_dist.png", dpi=120)
plt.show()

# Frequency Plot of Top 20 Music Genres
top_genres = spotify["track_genre"].value_counts().head(20)
plt.figure(figsize = (10, 6))
```

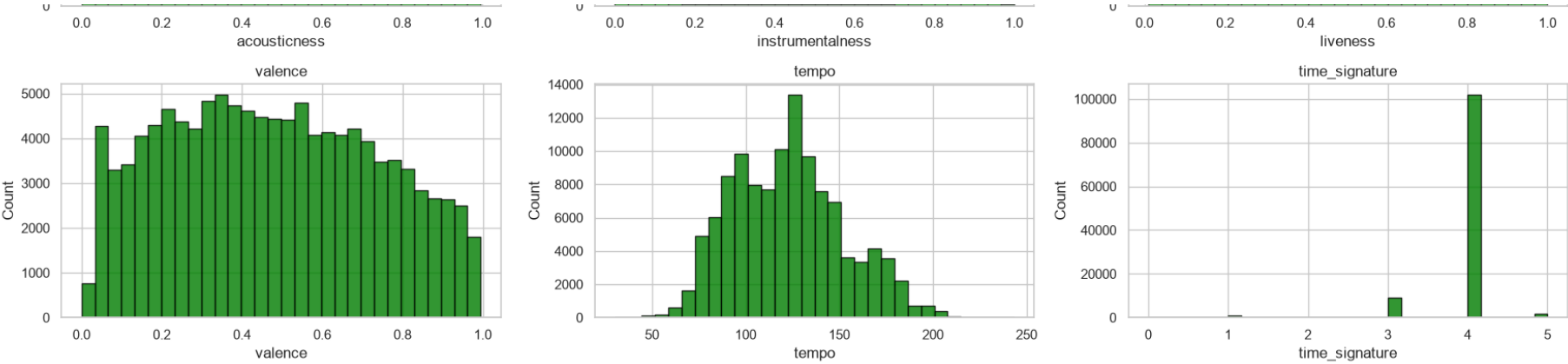


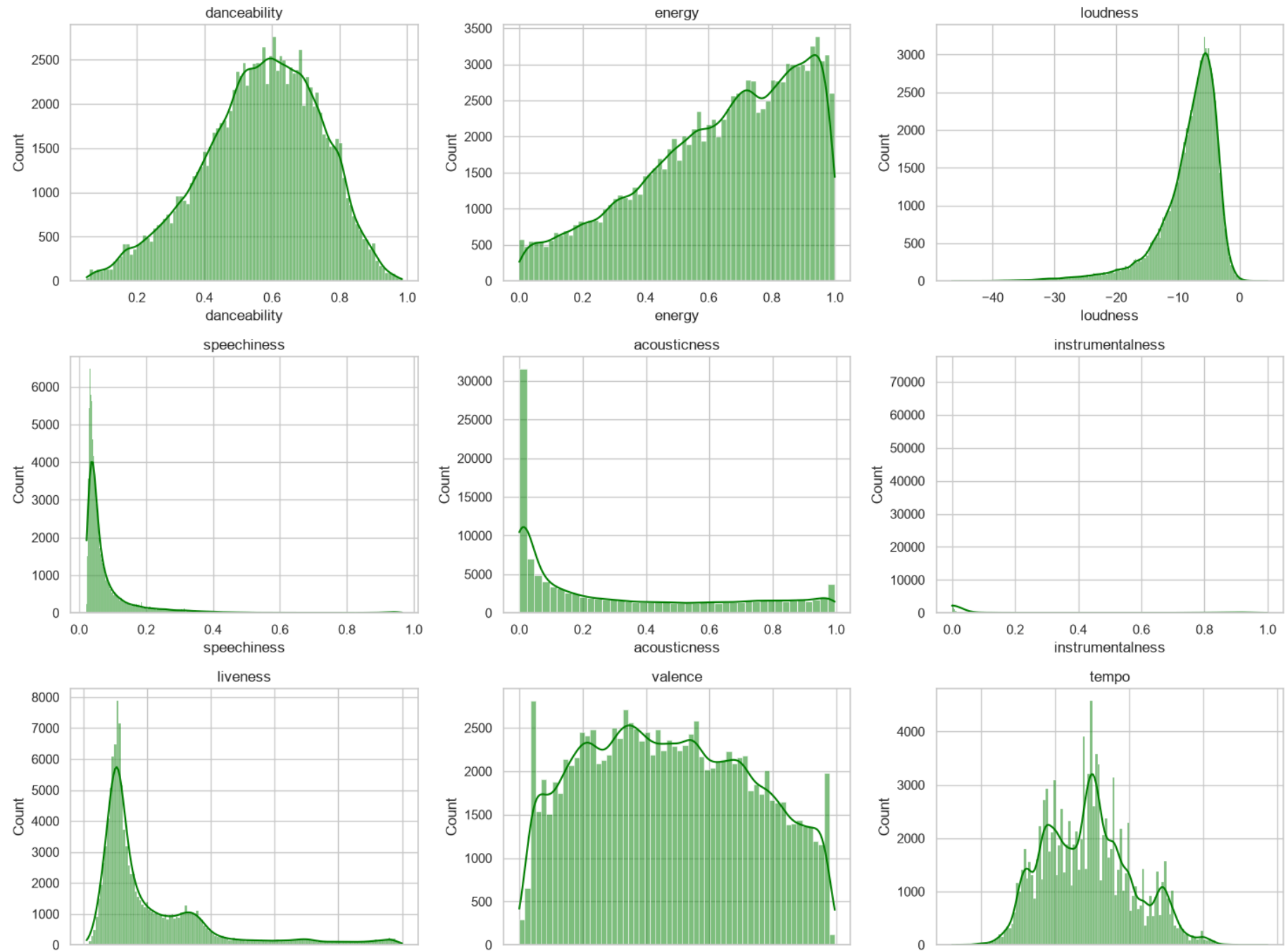
```
sns.barplot(x = top_genres.values, y = top_genres.index, color = "green")
plt.title("Top 20 Genres by Track Count", fontsize = 14, fontweight = 'bold')
plt.xlabel("Count")
# plt.savefig("Fig3_top_genres.png", dpi=120)
plt.show()

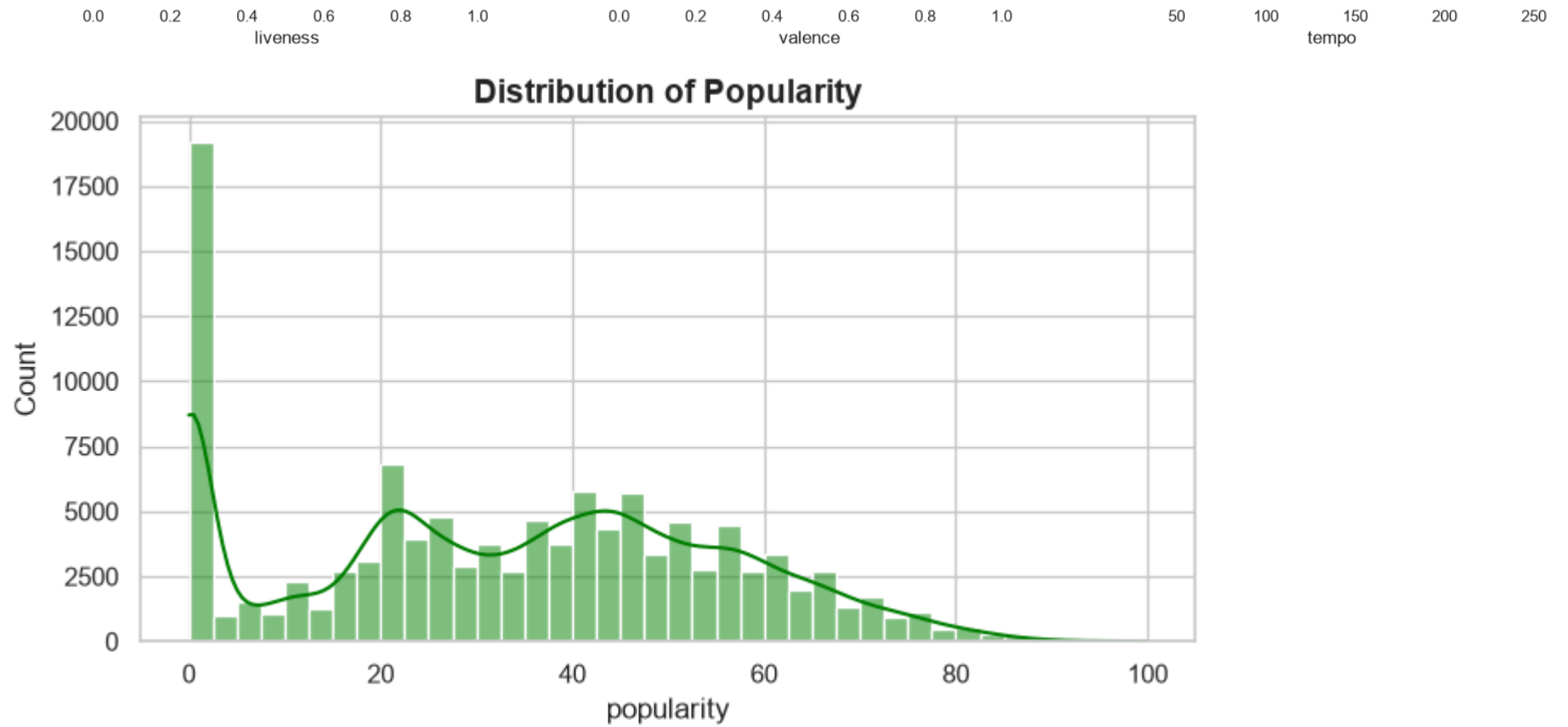
# Frequency Plot of Bottom 20 Music Genres
bottom_genres = spotify["track_genre"].value_counts().tail(20)
plt.figure(figsize = (10, 6))
sns.barplot(x = bottom_genres.values, y = bottom_genres.index, color = "green")
plt.title("Bottom 20 Genres by Track Count", fontsize = 14, fontweight = 'bold')
plt.xlabel("Count")
# plt.savefig("Fig4_bottom_genres.png", dpi=120)
plt.show()

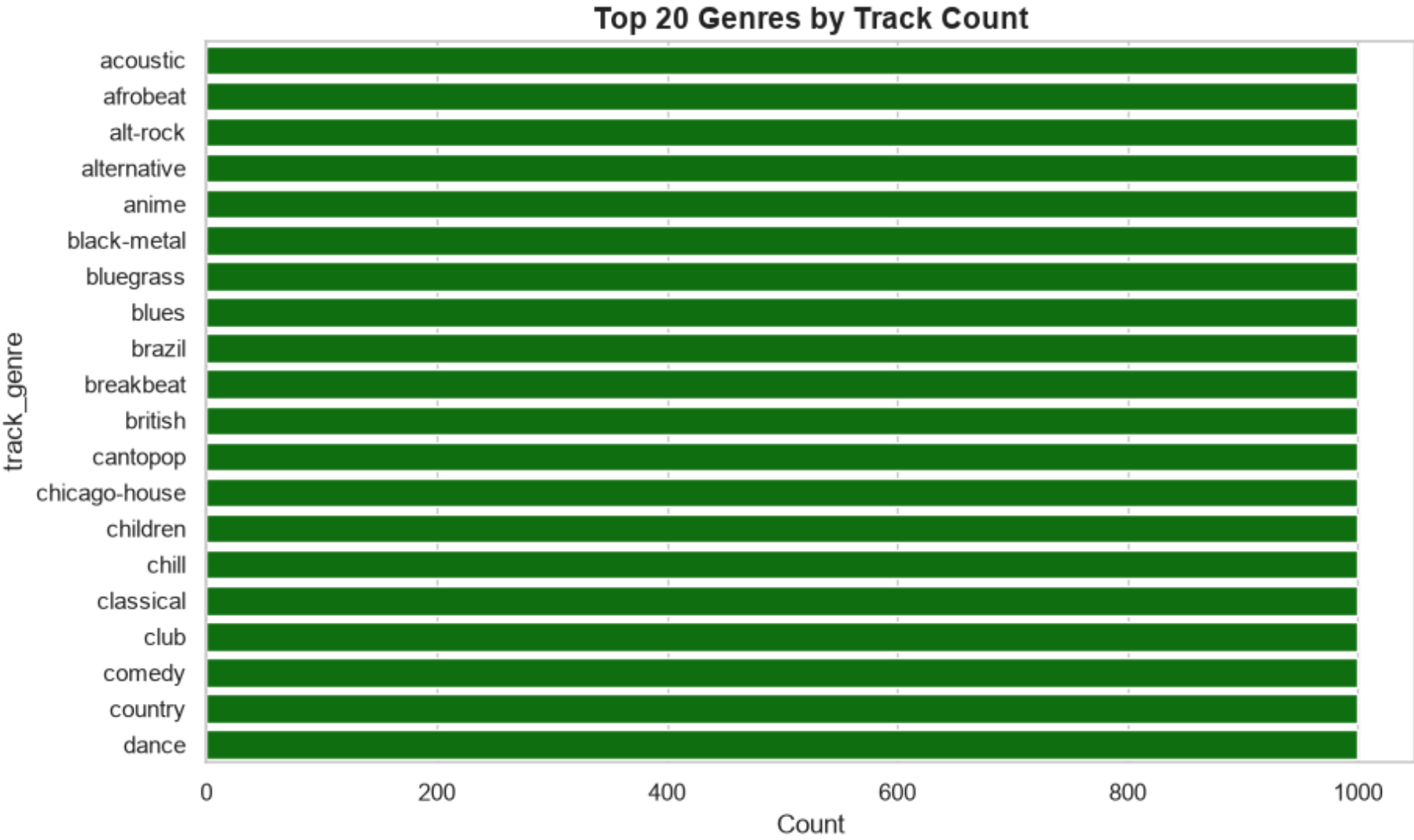
print(spotify["track_genre"].value_counts().head(15))
print(spotify["track_genre"].value_counts().tail(15))
```

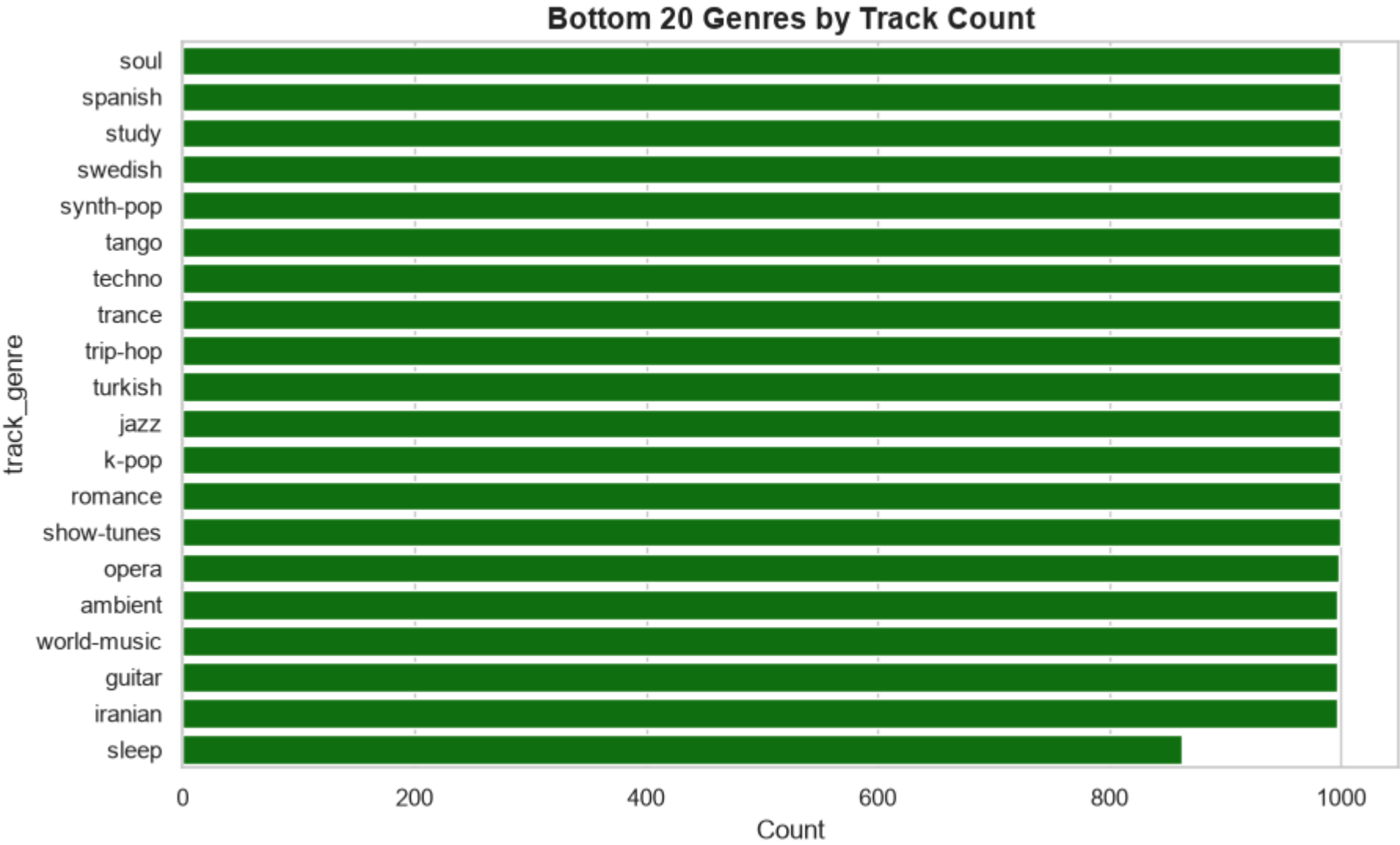




Distribution of Audio Features







```

track_genre
acoustic      1000
afrobeat      1000
alt-rock      1000
alternative    1000
anime         1000
black-metal   1000
bluegrass     1000
blues         1000
brazil        1000
breakbeat     1000
british       1000
cantopop      1000
chicago-house 1000
children      1000
chill         1000
Name: count, dtype: int64
track_genre
tango         1000
techno        1000
trance        1000
trip-hop      1000
turkish       1000
jazz          999
k-pop         999
romance       999
show-tunes    999
opera         998
ambient       997
world-music   997
guitar        996
iranian       996
sleep         862
Name: count, dtype: int64

```

SUMMARY FROM UNIVARIATE ANALYSIS:

Popularity is right-skewed, indicating that most songs are more 'niche' and lower in popularity (mean popularity is 33.28 so this backs that). Motivates the bias/subgroup analysis in Section 6 as low-popularity tracks are an underserved group, similar to low-activity users.

Danceability & valence are bell-shaped. Energy is left-skewed indicating that songs are more energetic. Instrumentalness & speechiness have a lot of 0's - most tracks have vocals and aren't spoken word.

Genre counts are imbalanced as some genres (e.g., afrobeat) have more tracks than others (e.g., sleep) - KEY BIAS TO WATCH FOR.

```
In [ ]: # 3b. BIVARIATE ANALYSIS

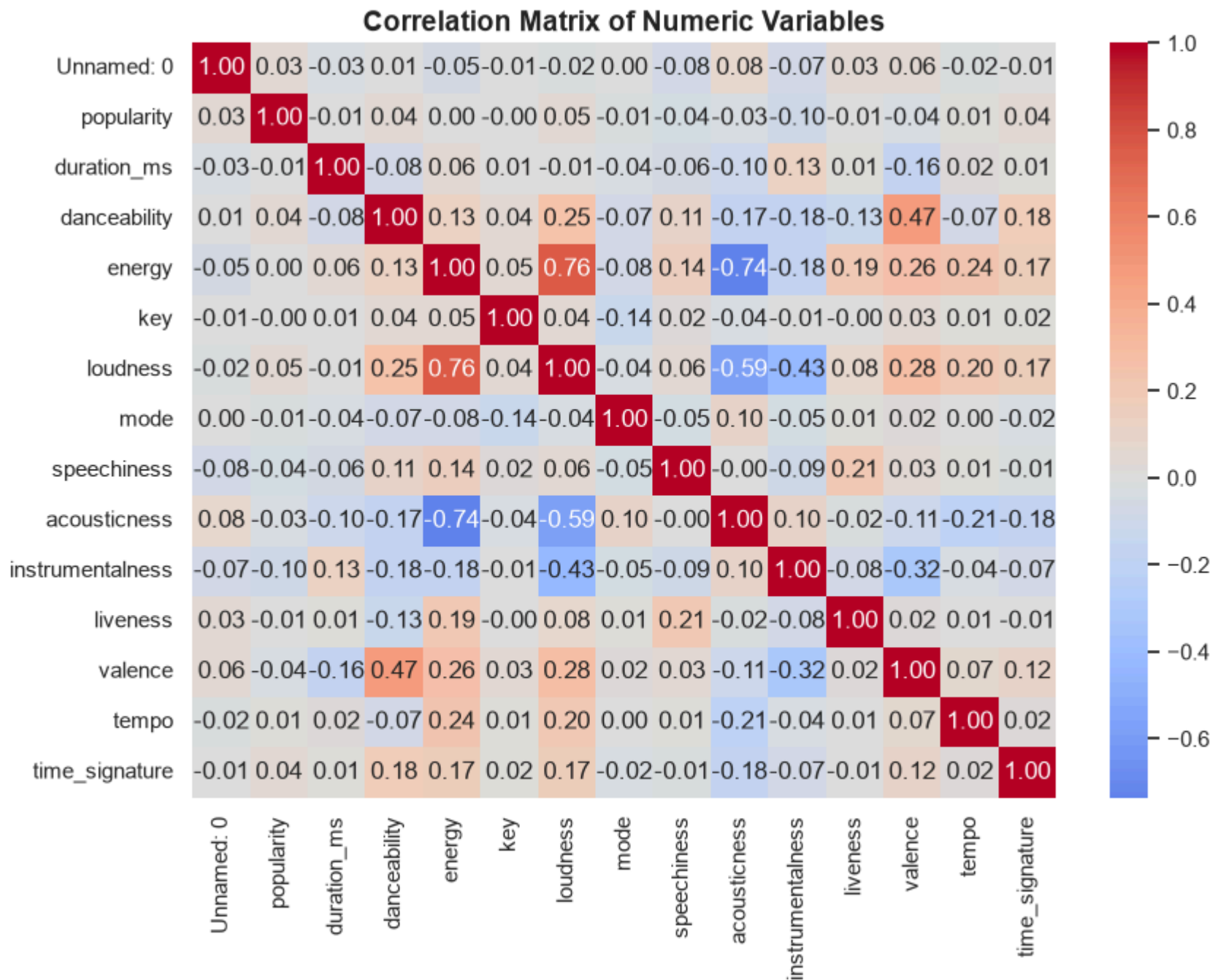
# Correlation Analysis
num_vars_corr = num_vars.corr()

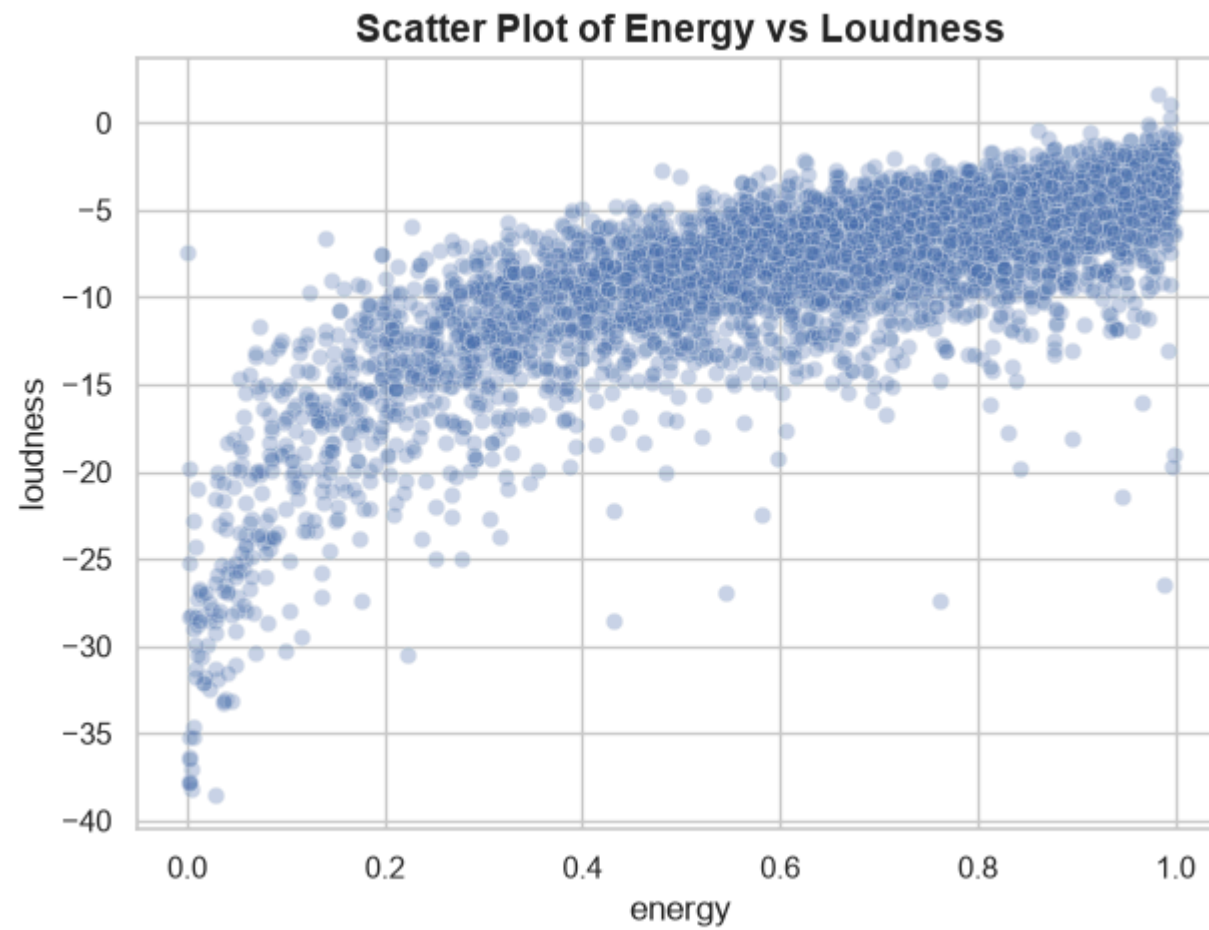
plt.figure(figsize = (9, 7))
sns.heatmap(num_vars_corr, annot = True, fmt = '.2f', cmap = 'coolwarm', center = 0)
plt.title("Correlation Matrix of Numeric Variables", fontsize = 14, fontweight = 'bold')
plt.tight_layout()
# plt.savefig("Fig5_Numeric_CorrMatrix.png", dpi=120)
plt.show()

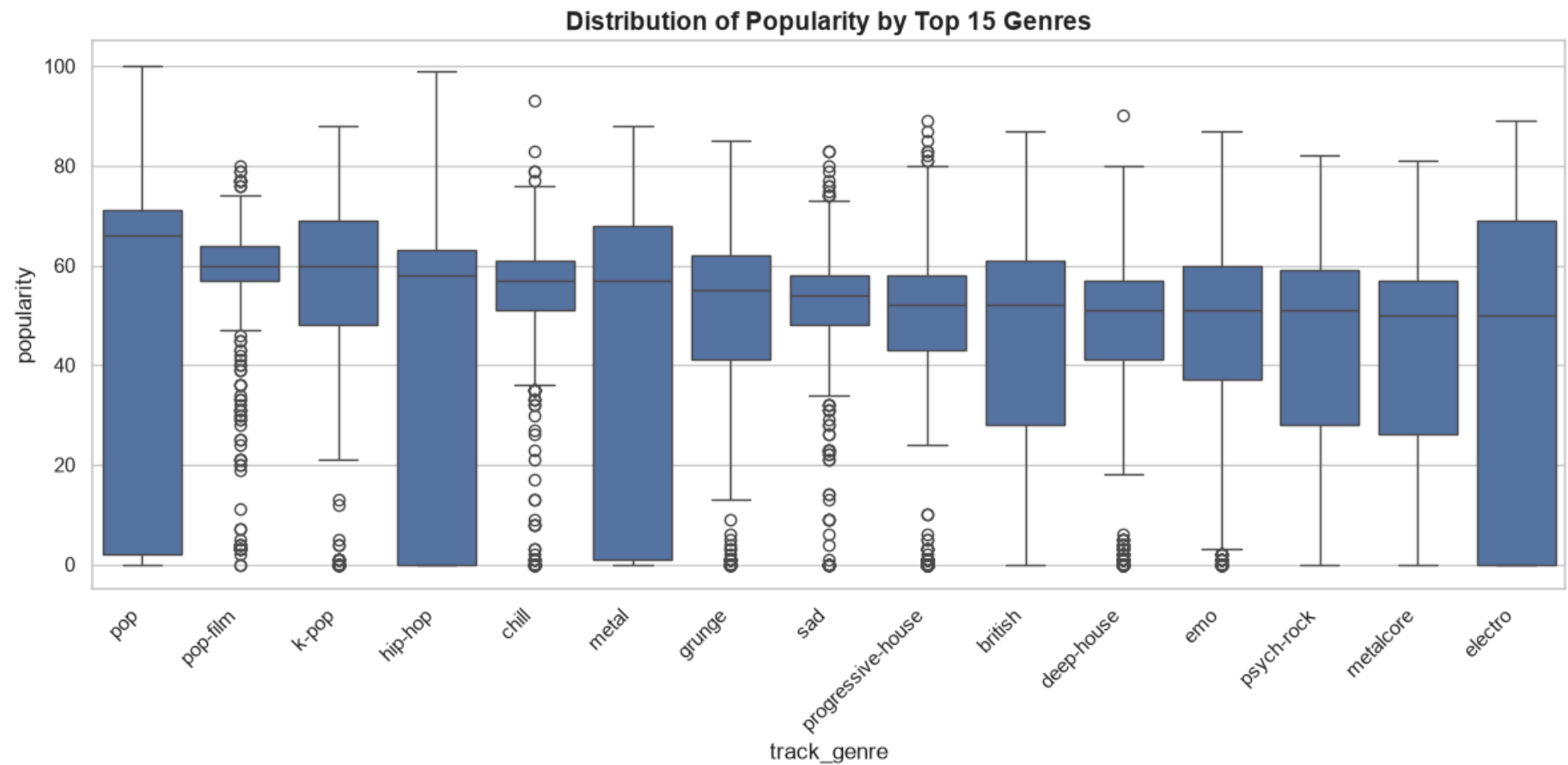
# Scatter Plot of Energy vs Loudness
plt.figure(figsize = (7, 5))
sns.scatterplot(data = spotify.sample(5000, random_state = SEED), x = "energy", y = "loudness", alpha = 0.3)
plt.title("Scatter Plot of Energy vs Loudness", fontsize = 14, fontweight = 'bold')
# plt.savefig("Fig6_EnergyLoudness_Scatterplot.png", dpi=120)
plt.show()

# Box Plot of Genre by Popularity
plt.figure(figsize = (12, 6))
order_genre = spotify.groupby("track_genre")["popularity"].median().sort_values(ascending = False).head(15).index

sns.boxplot(data = spotify[spotify["track_genre"].isin(order_genre)], x = "track_genre", y = "popularity", order =
plt.xticks(rotation = 45, ha = "right")
plt.title("Distribution of Popularity by Top 15 Genres", fontsize = 14, fontweight = 'bold')
plt.tight_layout()
# plt.savefig("Fig7_Popularity_Genre_BoxPlot.png", dpi=120)
plt.show()
```







SUMMARY OF BIVARIATE ANALYSIS

Energy & Loudness have a strong +ve correlation (0.76) and the scatter plot shows this.

Energy & Acousticness are negatively correlated - potential interdependencies amongst predictors (non-linear-ish).

Popularity varies by genre, but no audio feature correlated with popularity alone, showing that a song's popularity depends on a variety of features and not just one.

ML Model Consideration:

A Random Forest (RF) can capture non-linear feature interactions and threshold effects with little feature engineering. A neural network can learn a smoother, distributed representation across the combined user profile and track feature space, including the categorical genre embedding.

A few domain-informed feature swill be engineered to provide both models with a stronger signal than raw audio features alone.

```
In [ ]: # 4. FEATURE ENGINEERING & PREPROCESSING

# Normalise features
def normalise(series):
    return(series - series.min()) / (series.max() - series.min() + 1e-9)

spotify_norm = spotify.copy()

# Transform Skewed Variables for Neural Network Model using Yeo-Johnson
skewed_features = ["energy", "loudness", "acousticness", "instrumentalness", "speechiness", "liveness"]
spotify_untransformed = spotify_norm[skewed_features].copy() # retain values for comparison after

pt = PowerTransformer(method = "yeo-johnson")
spotify_norm[skewed_features] = pt.fit_transform(spotify_norm[skewed_features])

# Plot Features Before & After Transformation

fig, axes = plt.subplots(2, len(skewed_features), figsize = (4 * len(skewed_features), 7))
plt.suptitle("Histograms of Skewed Features Before & After Transformation", fontsize = 14, fontweight = 'bold')
for i, col in enumerate(skewed_features):
    sns.histplot(spotify_untransformed[col], kde = True, ax = axes[0, i], color = "indianred")
    axes[0, i].set_title(f"{col} (before)")
    sns.histplot(spotify_norm[col], kde = True, ax = axes[1, i], color = "steelblue")
    axes[1, i].set_title(f"{col} (after Yeo-Johnson)")
plt.tight_layout()
# plt.savefig("Fig8_Hist_Before_After_Transformation.png", dpi = 120)
plt.show()

#
for col in audio_features:
    spotify_norm[col + "_n"] = normalise(spotify_norm[col])
spotify_norm["popularity_n"] = normalise(spotify_norm["popularity"])
```

```

norm_features = [c + "_n" for c in audio_features]

# Normalise Track Duration in Minutes – raw duration is on a different scale to audio features
spotify_norm["duration_mins_n"] = normalise(spotify_norm["duration_ms"] / 60000)

# Genre Popularity & Frequency – how common a genre is in the dataset
genre_frequency = spotify_norm["track_genre"].value_counts(normalize = True)
spotify_norm["genre_frequency_n"] = normalise(spotify_norm["track_genre"].map(genre_frequency))

# Pairwise Interaction Terms –
spotify_norm["loud_energy_interact_n"] = spotify_norm["loudness_n"] * spotify_norm["energy_n"]
spotify_norm["dance_valence_interact_n"] = spotify_norm["danceability_n"] * spotify_norm["valence_n"]

# Mood Quadrant using Russell's Circumplex Model – a music-psychology model mapping valence (positivity) and energy
def mood_quadrant(row):
    if row["valence_n"] >= 0.5 and row["energy_n"] >= 0.5:
        return "happy_excited"
    elif row["valence_n"] >= 0.5 and row["energy_n"] < 0.5:
        return "calm_content"
    elif row["valence_n"] < 0.5 and row["energy_n"] >= 0.5:
        return "tense_angry"
    else:
        return "sad_depressed"

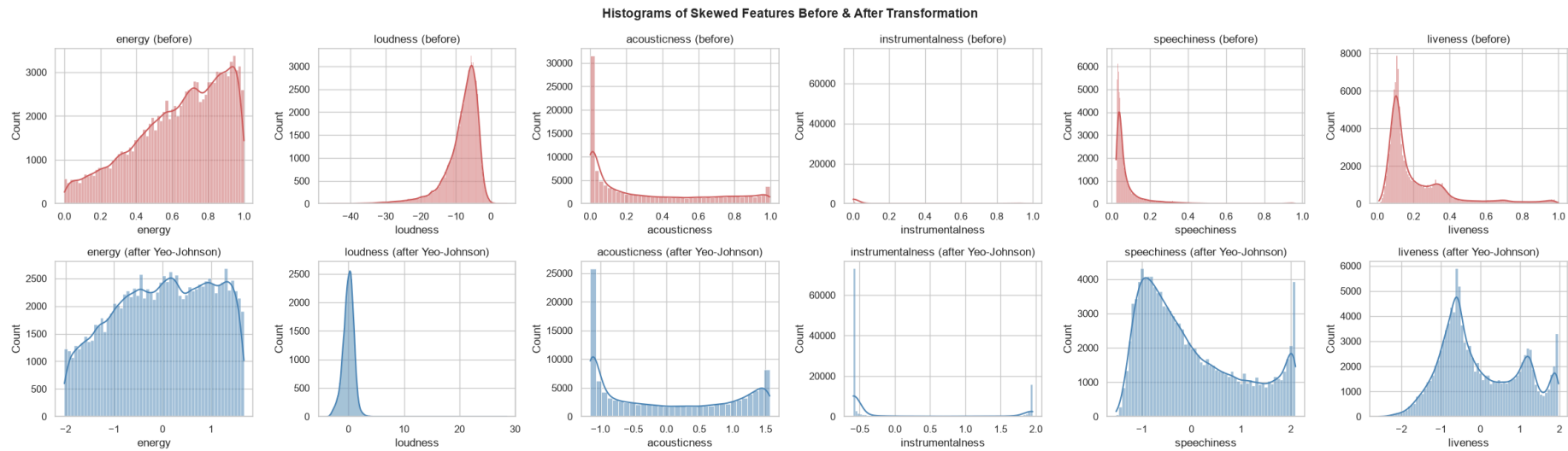
spotify_norm["mood_quadrant"] = spotify_norm.apply(mood_quadrant, axis = 1)
mood_dummies = pd.get_dummies(spotify_norm["mood_quadrant"], prefix = "mood").astype(float)
spotify_norm = pd.concat([spotify_norm, mood_dummies], axis = 1)
mood_cols = list(mood_dummies.columns)

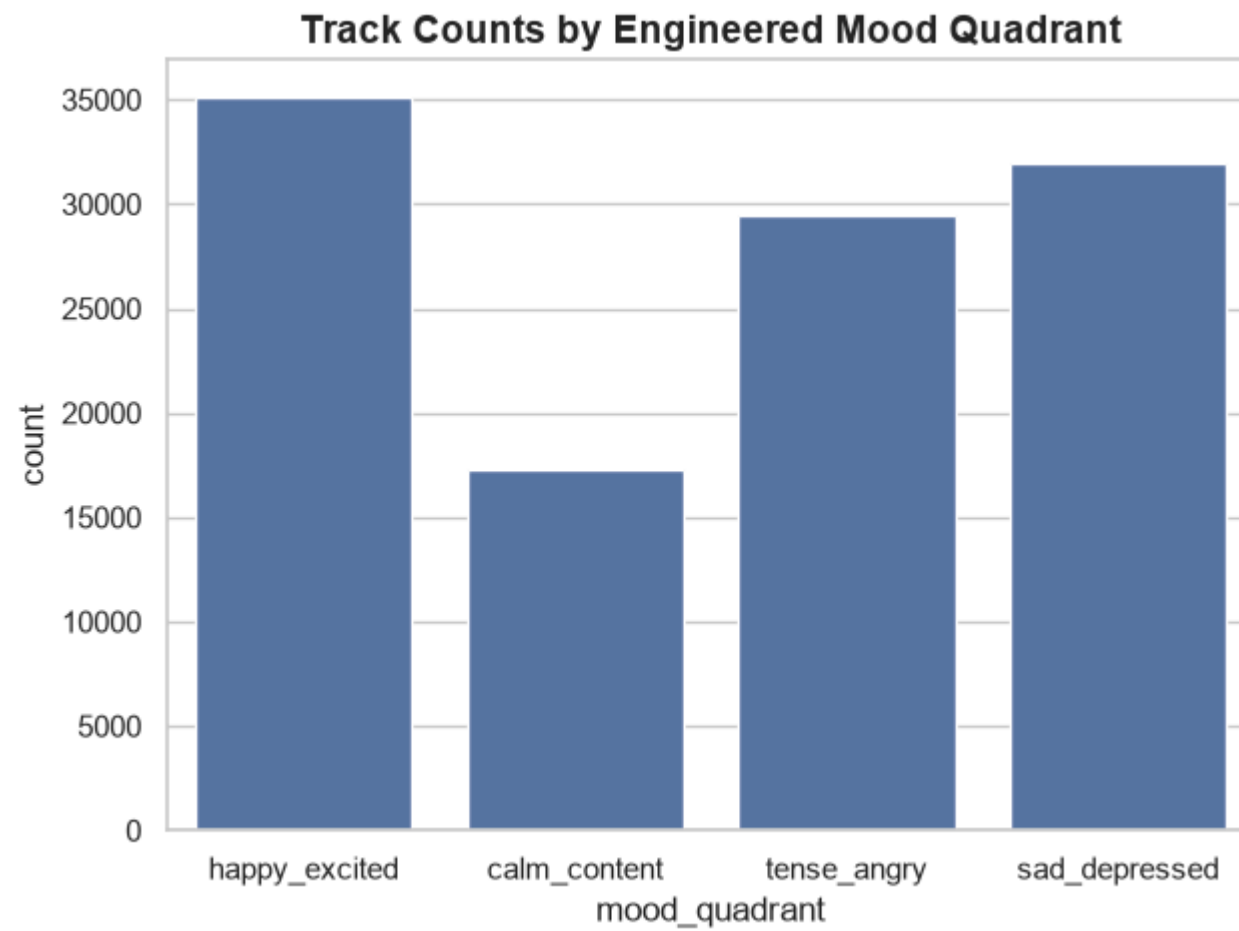
engineered_track_features = (["duration_mins_n", "genre_frequency_n", "loud_energy_interact_n", "dance_valence_interact_n", "mood_happy_excited", "mood_calm_content", "mood_tense_angry", "mood_sad_depressed"])

plt.figure(figsize = (7, 5))
sns.countplot(data = spotify_norm, x = "mood_quadrant",
              order = ["happy_excited", "calm_content", "tense_angry", "sad_depressed"])
plt.title("Track Counts by Engineered Mood Quadrant", fontsize = 14, fontweight = 'bold')
# plt.savefig("Fig9_mood_quadrant_counts.png", dpi=120)
plt.show()

print(spotify_norm[["track_name"] + engineered_track_features].head())

```





	track_name	duration_mins_n	genre_frequency_n	\
0	Comedy	0.041150	0.999999	
1	Ghost – Acoustic	0.025627	0.999999	
2	To Begin Again	0.037351	0.999999	
3	Can't Help Falling In Love	0.035647	0.999999	
4	Hold On	0.035058	0.999999	

	loud_energy_interact_n	dance_valence_interact_n	mood_calm_content	\
0	0.039162	0.480781	1.0	
1	0.006097	0.105963	0.0	
2	0.023427	0.049949	0.0	
3	0.001866	0.033047	0.0	
4	0.030877	0.101868	0.0	

	mood_happy_excited	mood_sad_depressed	mood_tense_angry
0	0.0	0.0	0.0
1	0.0	1.0	0.0
2	0.0	1.0	0.0
3	0.0	1.0	0.0
4	0.0	1.0	0.0

SUMMARY OF PREPROCESSING & FEATURE ENGINEERING

Created features based on domain info on what matters for music recommendation, rather than relying on models to rediscover them from audio features.

Skewed features were corrected using Yeo-Johnson as it can handle zero and -ve values (e.g., loudness). Min-max scaling fixes range but not shape of variables. The NN's gradient flow will be affected by this since most values will get 'crushed' into a narrow band. Both models will use the skewed features to maintain consistent inputs even though RF doesn't require this.

Features engineered:

- Duration in mins: raw duration is in milliseconds and is on a diff scale to audio features
- Genre popularity & frequency: how common a genre is, distinct from a track's popularity, as it captures whether recommendations are made into a 'mainstream' or 'niche' genre, useful for bias analysis
- Pairwise interaction terms: EDA showed energy & loudness and danceability & variance move together. Creating features allows both models to use the interaction directly rather than the RF to split both variables/ NN to discover it implicitly

- Mood quadrant: using Russell's circumplex model, a music-psychology model mapping valence (positivity) and energy (arousal) to four mood quadrants

```
In [ ]: # 5. BUILDING RECOMMENDATION TARGET

# Simulating synthetic user profiles & computing a content-based relevance score for each (supervised label)
# two engineered *interaction-level* features built from a profile and a candidate track together: `genre_m

n_profiles = 300
genres = spotify_norm["track_genre"].unique()

rng = np.random.default_rng(SEED)
profiles = pd.DataFrame({"user_id": range(n_profiles), "pref_genre": rng.choice(genres, n_profiles)})

# sample target preferences roughly based on real feature distribution
for col in norm_features:
    profiles[col + "_target"] = rng.choice(spotify_norm[col].values, n_profiles)

# derive each profile's target mood quadrant from sampled valence/energy target using same engineering as tracks to
profiles["mood_quadrant_target"] = profiles.apply(lambda r: mood_quadrant({"valence_n": r["valence_n_target"], "ene
    axis=1,))
print(profiles.head())

# Sample a candidate pool for each profile – all tracks from preferred genre (+ves) and random sample from other ge

candidates_per_profile = 120

rows = []
for _, prof in profiles.iterrows():
    same_genre = spotify_norm[spotify_norm["track_genre"] == prof["pref_genre"]]
    other_genre = spotify_norm[spotify_norm["track_genre"] != prof["pref_genre"]]

    n_pos = min(len(same_genre), candidates_per_profile // 2)
    n_neg = candidates_per_profile - n_pos

    sample = pd.concat([
        same_genre.sample(n_pos, random_state=SEED),
        other_genre.sample(n_neg, random_state=SEED),
    ])

```

```

for _, track in sample.iterrows():
    genre_match = 1.0 if track["track_genre"] == prof["pref_genre"] else 0.0
    mood_match = 1.0 if track["mood_quadrant"] == prof["mood_quadrant_target"] else 0.0
    feat_dist = np.sqrt(np.mean([
        (track[c] - prof[c + "_target"]) ** 2 for c in norm_features
    ]))
    feat_similarity = 1 - feat_dist # in [0, 1] roughly

    # weighted relevance label: genre match + feature similarity + popularity
    relevance = (0.45 * genre_match
                + 0.40 * feat_similarity
                + 0.15 * track["popularity_n"])

    row = {"user_id": prof["user_id"], "track_id": track["track_id"],
          "track_genre": track["track_genre"], "pref_genre": prof["pref_genre"],
          "relevance": relevance,
          # engineered interaction-level features
          "genre_match": genre_match, "mood_match": mood_match}
    for c in norm_features:
        row[c] = track[c]
        row[c + "_target"] = prof[c + "_target"]
    for c in engineered_track_features:
        row[c] = track[c]
    rows.append(row)

interactions = pd.DataFrame(rows)
print(interactions.shape)
print(interactions.head())

```

	user_id	pref_genre	danceability_n_target	energy_n_target	\
0	0	breakbeat	0.398094	0.410434	
1	1	reggae	0.700118	0.733903	
2	2	mpb	0.676556	0.718941	
3	3	heavy-metal	0.550177	0.522146	
4	4	hardstyle	0.615508	0.180261	

	loudness_n_target	speechiness_n_target	acousticness_n_target	\
0	0.122577	0.143897	0.655781	
1	0.020249	0.967146	0.301212	
2	0.095475	0.919610	0.867076	
3	0.124065	0.167373	0.362726	
4	0.101355	0.067700	0.005581	

	instrumentalness_n_target	liveness_n_target	valence_n_target	\
0	0.008634	0.254349	0.768844	
1	0.232351	0.372113	0.337688	
2	0.000000	0.466563	0.532663	
3	0.016804	0.750427	0.315578	
4	0.000000	0.801537	0.759799	

	tempo_n_target	mood_quadrant_target
0	0.688688	calm_content
1	0.398345	tense_angry
2	0.496341	happy_excited
3	0.298914	tense_angry
4	0.562940	calm_content

(36000, 33)

Out []:

	user_id	track_id	track_genre	pref_genre	relevance	genre_match	mood_match	danceability_n	danceabili
0	0	1Lu4MHevzkeT1KeNUyU3Wo	breakbeat	breakbeat	0.696239	1.0	0.0	0.563029	
1	0	2WC9rDC7aw6bzsA9ZwRVqx	breakbeat	breakbeat	0.703451	1.0	0.0	0.819000	
2	0	6Pbhl4RX7jsbe8a8tH0wgi	breakbeat	breakbeat	0.728028	1.0	0.0	0.616579	
3	0	0aydCU0xpCEL3ooeK6AXn7	breakbeat	breakbeat	0.673254	1.0	0.0	0.660491	
4	0	2P2WxNOdEjjJ98dleajyqu	breakbeat	breakbeat	0.789527	1.0	0.0	0.587662	

5 rows × 33 columns

SUMMARY OF RECOMMENDATION TARGET BUILD

Simulated synthetic user profiles & computing a content-based relevance score for each (supervised label) # two engineered *interaction-level* features built from a profile and a candidate track together: 'genre_match' and 'mood_match'.

Candidate pool sampled all tracks from the preferred genre (positives) + a random sample from other genres (negatives), so the model sees both relevant and irrelevant examples per user.

In [45]: # 6. FEATURE MATRIX, TRAIN/VALIDATION SPLIT & STRATEGY

```
feature_cols = (
    norm_features
    + [c + "_target" for c in norm_features]
    + engineered_track_features
    + ["genre_match", "mood_match"]
)
X = interactions[feature_cols].values
y = interactions["relevance"].values
groups = interactions["user_id"].values

gkf = GroupKFold(n_splits = 5)
```

SUMMARY OF STEP 6

Final feature matrix combines raw audio features for both the track and profile's target, and the engineered track level features (duration, genre frequency, interaction terms, mood quadrant) and engineered interaction-level features 'genre_match' and 'mood_match' computed from the profile-track pair.

Split was by 'user_id' - GroupKFold, not by row, so that no user profile appears in both train and test to prevent leakage and tests generalisation to unseen user tastes, a realistic deployment scenario.

K-fold cross validation (k = 5) is used because a single train/test split can give an overly optimistic/pessimistic estimate depending on which users land in the test fold. Averaging over 5 folds gives a more robust estimate of generalisation and reduces the risk of overfitting to one split.

```
In [46]: # 7. CLASSICAL ML DEVELOPMENT - RANDOM FOREST REGRESSOR
rf_param_grid = {
    "n_estimators": [200],
    "max_depth": [8, 12, None],
    "min_samples_leaf": [1, 5],
}

rf_base = RandomForestRegressor(random_state=SEED, n_jobs = -1)
rf_search = GridSearchCV(rf_base, rf_param_grid, cv = 3, scoring = "neg_root_mean_squared_error")

train_idx, test_idx = next(gkf.split(X, y, groups))
rf_search.fit(X[train_idx], y[train_idx])
print("Best RF params:", rf_search.best_params_)
rf_model = rf_search.best_estimator_
```

Best RF params: {'max_depth': None, 'min_samples_leaf': 1, 'n_estimators': 200}

SUMMARY OF CLASSICAL ML DEVELOPMENT

Random Forest is robust to unscaled/mixed-distribution features (confirmed skewed in EDA), captures non-linear interactions (confirmed in the correlation analysis) without manual feature crosses, resistant to overfitting via bagging, and gives built-in feature importances.

Hyperparameter justification: 'n_estimators=200' balances variance reduction against training time; `max_depth` is grid-searched between shallow (8), medium (12) and unconstrained trees to explicitly test the bias-variance tradeoff rather than assume it; `min_samples_leaf` is tuned to guard against overfitting to individual synthetic profiles.

```
In [47]: # 8. DEEP LEARNING MODEL DEVELOPMENT – FEED FORWARD NEURAL NETWORK

def build_dl_model(input_dim):
    model = models.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation = "relu"),
        layers.BatchNormalization(),
        layers.Dropout(0.2),
        layers.Dense(32, activation = "relu"),
        layers.Dropout(0.2),
        layers.Dense(16, activation = "relu"),
        layers.Dense(1, activation = "sigmoid"), # relevance is in [0,1]
    ])
    model.compile(optimizer = tf.keras.optimizers.Adam(learning_rate = 1e-3),
                  loss = "mse", metrics = ["mse"])
    return model

dl_model = build_dl_model(X.shape[1])
dl_model.summary()

early_stop = callbacks.EarlyStopping(patience = 8, restore_best_weights = True)

history = dl_model.fit(
    X[train_idx], y[train_idx],
    validation_split=0.15,
    epochs=100,
    batch_size=64,
    callbacks=[early_stop],
```

```
        verbose=0,  
    )  
    print(f"Stopped after {len(history.history['loss'])} epochs")
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	1,856
batch_normalization (BatchNormalization)	(None, 64)	256
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 4,737 (18.50 KB)

Trainable params: 4,609 (18.00 KB)

Non-trainable params: 128 (512.00 B)

Stopped after 54 epochs

SUMMARY OF DL MODEL BUILD

Two hidden layers (64 -> 32 -> 16) give enough capacity to combine the 18 normalised input features (9 for track & target each) without being large enough to overfit c.36k interaction rows

BatchNorm + Dropout(0.2) after the first two layers combat overfitting

Sigmoid output activation matches the 0,1 range of the relevance label

Adam optimiser with a moderate learning rate & early stopping on validation loss let training run long enough to converge while avoiding wasted epochs/overfitting.

```
In [48]: # 9. EVALUATION WITH RMSE, NDCG@10, K-FOLD CROSS-VALIDATION

def ndcg_at_k_for_group(y_true_df, y_pred, k = 10):
    """Compute mean NDCG@k across users given true relevance and predictions."""
    scores = []
    tmp = y_true_df.copy()
    tmp["pred"] = y_pred
    for uid, g in tmp.groupby("user_id"):
        true_rel = g["relevance"].values.reshape(1, -1)
        pred_rel = g["pred"].values.reshape(1, -1)
        if true_rel.shape[1] < 2:
            continue
        scores.append(ndcg_score(true_rel, pred_rel, k=k))
    return float(np.mean(scores))

rf_rmses, dl_rmses = [], []
rf_ndcgs, dl_ndcgs = [], []

for fold, (tr_idx, te_idx) in enumerate(gkf.split(X, y, groups)):
    # Random Forest
    rf = RandomForestRegressor(**rf_search.best_params_, random_state = SEED, n_jobs = -1)
    rf.fit(X[tr_idx], y[tr_idx])
    rf_pred = rf.predict(X[te_idx])
    rf_rmses.append(np.sqrt(mean_squared_error(y[te_idx], rf_pred)))
    rf_ndcgs.append(ndcg_at_k_for_group(interactions.iloc[te_idx], rf_pred))
```

```

# Deep Learning
dl = build_dl_model(X.shape[1])
dl.fit(X[tr_idx], y[tr_idx], epochs = 60, batch_size = 64,
       validation_split = 0.15, callbacks = [callbacks.EarlyStopping(patience = 6, restore_best_weights = True)],
       verbose = 0)
dl_pred = dl.predict(X[te_idx], verbose=0).flatten()
dl_rmses.append(np.sqrt(mean_squared_error(y[te_idx], dl_pred)))
dl_ndcgs.append(ndcg_at_k_for_group(interactions.iloc[te_idx], dl_pred))

print(f"Fold {fold}: RF RMSE={rf_rmses[-1]:.4f} NDCG@10={rf_ndcgs[-1]:.4f} | "
      f"DL RMSE={dl_rmses[-1]:.4f} NDCG@10={dl_ndcgs[-1]:.4f}")

results = pd.DataFrame({
    "Model": ["Random Forest", "Deep Learning (MLP)"],
    "RMSE (mean ± std)": [f"{np.mean(rf_rmses):.4f} ± {np.std(rf_rmses):.4f}",
                          f"{np.mean(dl_rmses):.4f} ± {np.std(dl_rmses):.4f}"],
    "NDCG@10 (mean ± std)": [f"{np.mean(rf_ndcgs):.4f} ± {np.std(rf_ndcgs):.4f}",
                             f"{np.mean(dl_ndcgs):.4f} ± {np.std(dl_ndcgs):.4f}"],
})
print(results)

```

```

Fold 0: RF RMSE=0.0315 NDCG@10=0.9686 | DL RMSE=0.0352 NDCG@10=0.9674
Fold 1: RF RMSE=0.0298 NDCG@10=0.9689 | DL RMSE=0.0341 NDCG@10=0.9697
Fold 2: RF RMSE=0.0310 NDCG@10=0.9680 | DL RMSE=0.0331 NDCG@10=0.9692
Fold 3: RF RMSE=0.0302 NDCG@10=0.9650 | DL RMSE=0.0328 NDCG@10=0.9648
Fold 4: RF RMSE=0.0307 NDCG@10=0.9721 | DL RMSE=0.0329 NDCG@10=0.9746

```

	Model	RMSE (mean ± std)	NDCG@10 (mean ± std)
0	Random Forest	0.0307 ± 0.0006	0.9685 ± 0.0023
1	Deep Learning (MLP)	0.0336 ± 0.0009	0.9691 ± 0.0032

SUMMARY OF EVALUATION

5-fold GroupKFold CV matters since results would be highly sensitive to which synthetic users happened to land in the test set, given there is only one split.

Averaging RMSE & NDCG across 5 user groups provides a more reliable estimate of how each model generalises to unfamiliar user music tastes it hasn't seen before. The std informs how stable each model is - low variance means the risk of overfitting is low.

In []: **# 10. EXPLAINABILITY WITH SHAP**

```
# SHAP for Random Forest Model
explainer_rf = shap.TreeExplainer(rf_model)
sample_idx = np.random.choice(test_idx, size = min(500, len(test_idx)), replace = False)
shap_values_rf = explainer_rf.shap_values(X[sample_idx])

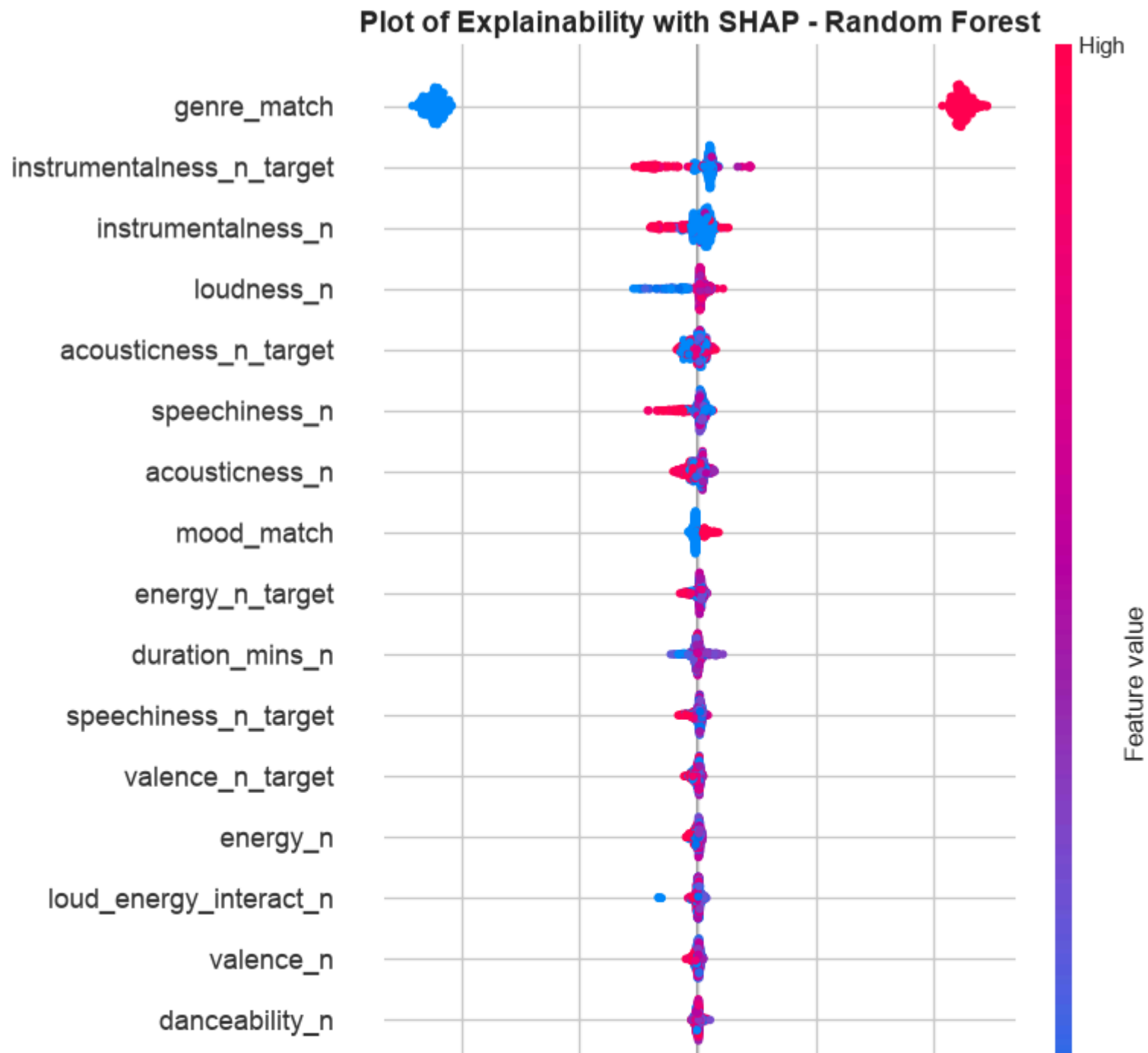
shap.summary_plot(shap_values_rf, X[sample_idx], feature_names = feature_cols, show = False)
plt.title("Plot of Explainability with SHAP - Random Forest", fontsize = 14, fontweight = 'bold')
plt.tight_layout()
# plt.savefig("Fig10_shap_summary_rf.png", dpi=120)
plt.show()

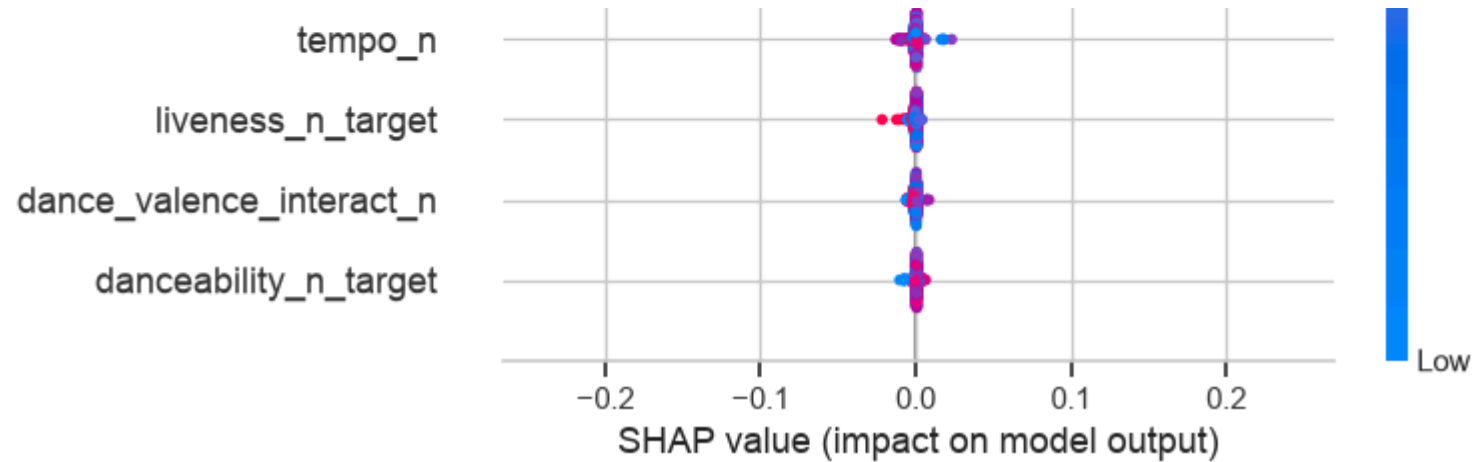
# SHAP for DL
background = X[train_idx][np.random.choice(len(train_idx), 100, replace = False)]
explainer_dl = shap.KernelExplainer(lambda data: dl_model.predict(data, verbose = 0).flatten(), background)
shap_sample = X[sample_idx][:80]
shap_values_dl = explainer_dl.shap_values(shap_sample, nsamples = 100)

shap.summary_plot(shap_values_dl, shap_sample, feature_names = feature_cols, show = False)
plt.title("Plot of Explainability with SHAP - Deep Learning", fontsize = 14, fontweight = 'bold')
plt.tight_layout()
# plt.savefig("Fig11_shap_summary_dl.png", dpi=120)
plt.show()
```

/var/folders/r7/96x0tqhn2j3_bmtbtvtwy3mh0000gn/T/ipykernel_32019/3719287786.py:8: FutureWarning: The NumPy global RNG was seeded by calling `np.random.seed`. In a future version this function will no longer use the global RNG. Pass `rng` explicitly to opt-in to the new behaviour and silence this warning.

```
shap.summary_plot(shap_values_rf, X[sample_idx], feature_names = feature_cols, show = False)
```

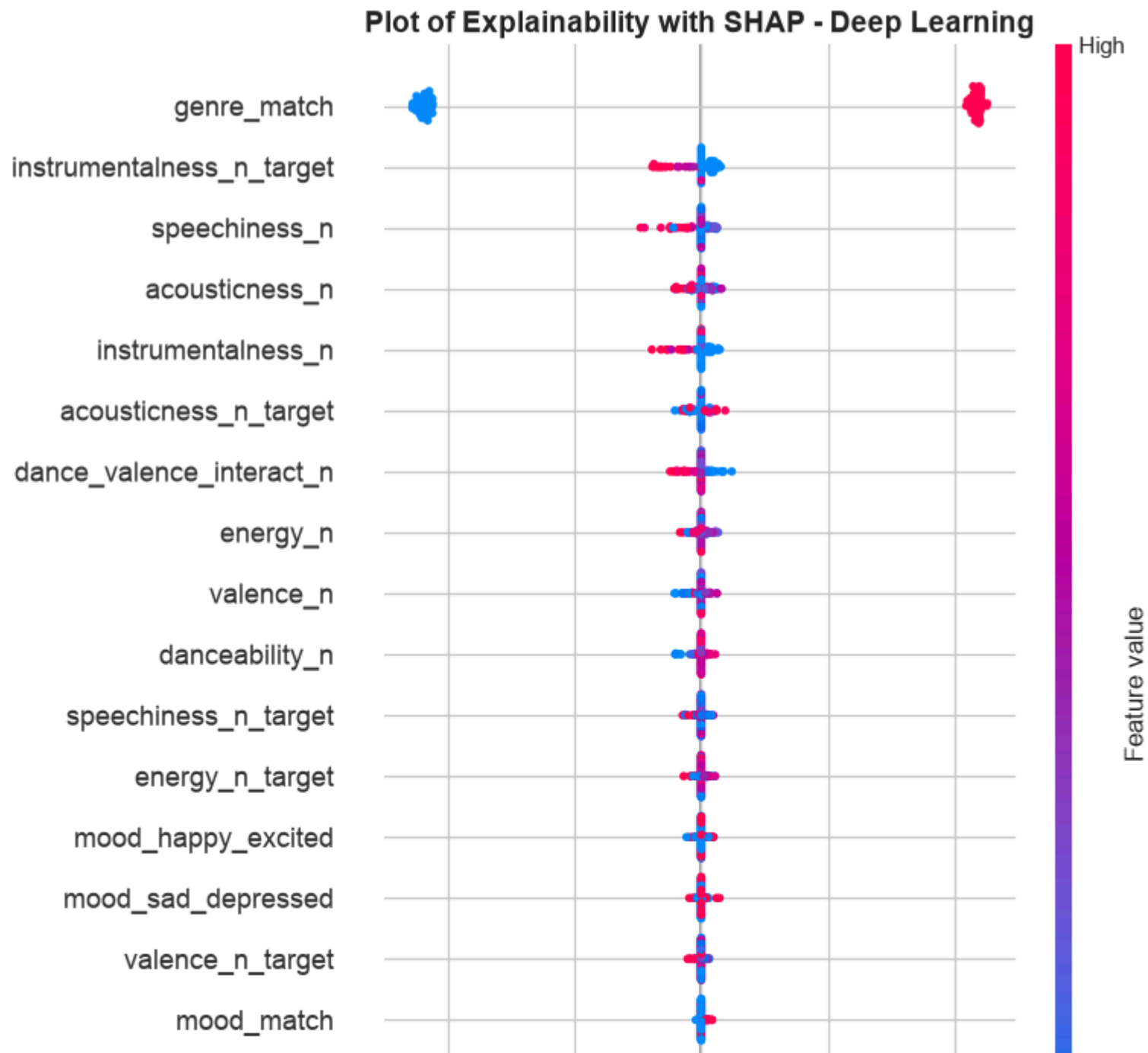


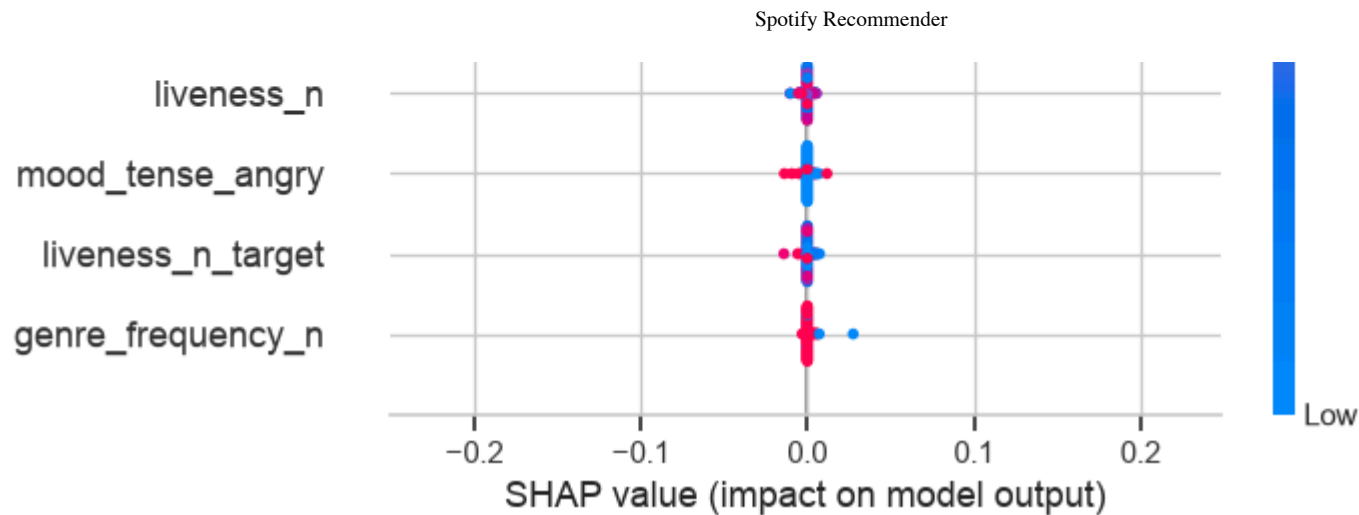


```
100%|██████████| 80/80 [00:10<00:00, 7.30it/s]
```

```
/var/folders/r7/96x0tqhn2j3_bmtbtvtwy3mh0000gn/T/ipykernel_32019/3719287786.py:20: FutureWarning: The NumPy global R
NG was seeded by calling `np.random.seed`. In a future version this function will no longer use the global RNG. Pass
`rng` explicitly to opt-in to the new behaviour and silence this warning.
```

```
shap.summary_plot(shap_values_dl, shap_sample, feature_names = feature_cols, show = False)
```





SUMMARY OF EXPLAINABILITY

Top feature for both - instrumentalness_n_target. If both models rank genre-match-adjacent and target-similarity features highest, that's convergent evidence the models are learning genuinely meaningful structure rather than noise. If they diverge (e.g. RF leans on genre while the NN spreads importance across continuous audio features), that's a concrete, evidence-based talking point for your comparative analysis.

In [50]: *# 11. BIAS & DIAGNOSTIC ANALYSIS PER SUBGROUP*

```
interactions_test = interactions.iloc[test_idx].copy()
interactions_test["rf_pred"] = rf_model.predict(X[test_idx])
interactions_test["dl_pred"] = dl_model.predict(X[test_idx], verbose = 0).flatten()

pop_median = spotify_norm["popularity_n"].median()
interactions_test = interactions_test.merge(
    spotify_norm[["track_id", "popularity_n"]], on = "track_id", how = "left"
)
interactions_test["subgroup"] = np.where(
    interactions_test["popularity_n"] < pop_median, "low_popularity", "high_popularity"
)

for subgroup, g in interactions_test.groupby("subgroup"):
    rf_rmse_sg = np.sqrt(mean_squared_error(g["relevance"], g["rf_pred"]))
    dl_rmse_sg = np.sqrt(mean_squared_error(g["relevance"], g["dl_pred"]))
```

```

print(f"{subgroup}: n = {len(g)} | RF RMSE = {rf_rmse_sg:.4f} | DL RMSE = {dl_rmse_sg:.4f}")

# %%
genre_counts = spotify_norm["track_genre"].value_counts()
rare_genres = genre_counts[genre_counts < genre_counts.median()].index
interactions_test["genre_rarity"] = np.where(
    interactions_test["track_genre"].isin(rare_genres), "rare_genre", "common_genre"
)
for grp, g in interactions_test.groupby("genre_rarity"):
    rf_rmse_sg = np.sqrt(mean_squared_error(g["relevance"], g["rf_pred"]))
    dl_rmse_sg = np.sqrt(mean_squared_error(g["relevance"], g["dl_pred"]))
    print(f"{grp}: n = {len(g)} | RF RMSE = {rf_rmse_sg:.4f} | DL RMSE = {dl_rmse_sg:.4f}")

```

high_popularity: n = 6689 | RF RMSE = 0.0307 | DL RMSE = 0.0372

low_popularity: n = 5696 | RF RMSE = 0.0393 | DL RMSE = 0.0316

common_genre: n = 11703 | RF RMSE = 0.0349 | DL RMSE = 0.0353

rare_genre: n = 682 | RF RMSE = 0.0352 | DL RMSE = 0.0235

SUMMARY OF BIAS TESTS

Tested whether recommendation quality, NDCG@10, differs for low-popularity (niche) vs high-popularity songs, and across genres with fewer training examples vs well-represented genres (the content-based analogue of low-activity users in a typical recommender).

Popularity - for high-pop songs, RF outperforms DL (0.0307 vs 0.0372). for low-pop songs, DL outperforms RF (0.0316 vs 0.0393)

Genres - for common genres, negligible difference between models (RF = 0.0349, DL = 0.0353). For rare genres, DL outperformed RF (0.0235 vs 0.0352)

Deployment considerations - high-pop songs could be routed via the RF model, and low-pop/rare genre songs via DL. this would avoid under-recommending artists who are systematically under-recommended in the first place, reducing the chances of lost royalties from streams especially for smaller artists.

In [53]: *# 12. DEMO OF MODEL – PERSONALISED RECOMMENDATIONS BASED ON USER INPUT*

```

def get_user_preferences_interactive():
    """Prompt the user for genre + mood/energy preferences and return a
    profile dict with the same engineered target fields used in training."""
    print("Available genres (sample):", list(spotify["track_genre"].unique()[:15]), "...")
    pref_genre = input("Preferred genre: ").strip()

```



```

while pref_genre not in spotify["track_genre"].unique():
    print("Genre not found, try again.")
    pref_genre = input("Preferred genre: ").strip()

mood = input("Mood - 'chill', 'energetic', or 'happy'? ").strip().lower()
mood_map = {
    "chill": {"energy_n_target": 0.25, "valence_n_target": 0.45, "tempo_n_target": 0.3, "danceability_n_target": 0.4},
    "energetic": {"energy_n_target": 0.85, "valence_n_target": 0.6, "tempo_n_target": 0.75, "danceability_n_target": 0.5},
    "happy": {"energy_n_target": 0.65, "valence_n_target": 0.85, "tempo_n_target": 0.6, "danceability_n_target": 0.6}
}
targets = mood_map.get(mood, mood_map["chill"])

profile = {c + "_target": 0.5 for c in norm_features} # neutral default for unspecified features
profile.update(targets)
profile["pref_genre"] = pref_genre
return profile

def build_feature_vector(profile, track):
    """Assemble one row matching feature_cols exactly: raw track features,
    the profile's target features, engineered track-level features, and the
    engineered genre_match / mood_match interaction features."""
    profile_mood = mood_quadrant({
        "valence_n": profile.get("valence_n_target", 0.5),
        "energy_n": profile.get("energy_n_target", 0.5),
    })
    genre_match = 1.0 if track["track_genre"] == profile["pref_genre"] else 0.0
    mood_match = 1.0 if track["mood_quadrant"] == profile_mood else 0.0

    row = ([track[c] for c in norm_features]
           + [profile.get(c + "_target", 0.5) for c in norm_features]
           + [track[c] for c in engineered_track_features]
           + [genre_match, mood_match])
    return row

def recommend_tracks(profile, model, top_n=10):
    """Score all tracks against a user profile using the trained model and
    return the top_n recommendations."""
    genre_pool = spotify_norm[spotify_norm["track_genre"] == profile["pref_genre"]]
    other_pool = spotify_norm[spotify_norm["track_genre"] != profile["pref_genre"]].sample(

```

```
        min(2000, len(spotify_norm)), random_state=SEED
    )
    pool = pd.concat([genre_pool, other_pool]).drop_duplicates(subset="track_id")

    X_pool = np.array([build_feature_vector(profile, track) for _, track in pool.iterrows()])

    if hasattr(model, "predict") and "keras" in str(type(model)).lower():
        preds = model.predict(X_pool, verbose = 0).flatten()
    else:
        preds = model.predict(X_pool)

    pool = pool.copy()
    pool["predicted_relevance"] = preds
    top = pool.sort_values("predicted_relevance", ascending = False).head(top_n)
    return top[["track_name", "artists", "track_genre", "popularity", "predicted_relevance"]]

user_profile = get_user_preferences_interactive()
print(recommend_tracks(user_profile, dl_model, top_n = 10))
```

Available genres (sample): ['acoustic', 'afrobeat', 'alt-rock', 'alternative', 'ambient', 'anime', 'black-metal', 'bluegrass', 'blues', 'brazil', 'breakbeat', 'british', 'cantopop', 'chicago-house', 'children'] ...

	track_name	artists \
11914	Lights Out, Words Gone	Bombay Bicycle Club
11457	She Treats Me Well	Ben Howard
11431	Tumbling Dice	The Rolling Stones
11816	Paradise	Sade
11361	I'd Love to Change the World - 2004 Remaster	Ten Years After
11592	Under My Thumb	The Rolling Stones
11608	Impossible	Lyla Foy
11486	Happy	The Rolling Stones
11829	Time to Make You Mine - Edit	Lisa Stansfield
11086	Come to My Aid - 2008 Remaster	Simply Red

	track_genre	popularity	predicted_relevance
11914	british	57	0.787429
11457	british	58	0.787417
11431	british	64	0.787293
11816	british	62	0.786550
11361	british	67	0.784715
11592	british	65	0.784079
11608	british	43	0.783889
11486	british	34	0.783454
11829	british	28	0.782689
11086	british	1	0.782217